

Using *continue* in a Loop

Rather than breaking out of a loop entirely without executing the rest of its code, you can use the `continue` statement to return to the beginning of the loop, based on the result of a conditional test. For example, consider a loop that counts from 1 to 10 but prints only the odd numbers in that range:

```
counting.py current_number = 0
while current_number < 10:
    ❶ current_number += 1
      if current_number % 2 == 0:
          continue

      print(current_number)
```

First, we set `current_number` to 0. Because it's less than 10, Python enters the `while` loop. Once inside the loop, we increment the count by 1 ❶, so `current_number` is 1. The `if` statement then checks the modulo of `current_number` and 2. If the modulo is 0 (which means `current_number` is divisible by 2), the `continue` statement tells Python to ignore the rest of the loop and return to the beginning. If the current number is not divisible by 2, the rest of the loop is executed and Python prints the current number:

```
1
3
5
7
9
```

Avoiding Infinite Loops

Every `while` loop needs a way to stop running so it won't continue to run forever. For example, this counting loop should count from 1 to 5:

```
counting.py x = 1
while x <= 5:
    print(x)
    x += 1
```

However, if you accidentally omit the line `x += 1`, the loop will run forever:

```
# This loop runs forever!
x = 1
while x <= 5:
    print(x)
```
